UNIX Shell with Priority-Based Batch Scheduling

1. Project Title & Team

Project: G.2 UNIX Shell

Title: UNIX Shell with Priority-Based Batch Scheduling

Course: Computer Systems

Team Size: 4 members

Team Members:
- Zihan Guo
- Changming Qiu
- Parker McKillop
- Rowan Lowden

2. Abstract

This project aims to develop a UNIX shell that supports basic command execution, pipelines, input/output redirection, and process management. The shell will use standard UNIX process APIs, including fork(), execvp(), waitpid(), pipe(), and dup2(). In addition to these standard features, we propose a priority-based batch scheduling mechanism that allows users to submit jobs with different priority levels. We will implement two scheduling policies, First-In-First-Out (FIFO) and priority-based scheduling, and compare their behavior using identical workloads. Our main goal is to investigate whether priority-based scheduling can reduce the waiting time of high-priority jobs and understand how this affects lower-priority jobs. We will measure waiting time, turnaround time, and job completion order to evaluate the benefits and trade-offs of the two scheduling policies.

3. Research Extension (Hypothesis)

Our research extension is a priority-based batch scheduler implemented inside our UNIX shell.

The baseline implementation will use a First-In-First-Out (FIFO) queue. Jobs will be executed in the order they are submitted, regardless of their priority levels.

The extended implementation will use a priority queue implemented with a binary heap. Each job will have a priority level and a submission order number. The scheduler will select the highest-priority job first. When multiple jobs have the same priority, the job submitted earlier will be selected first.

We hypothesize that priority-based scheduling will reduce the average waiting time of high-priority jobs compared with FIFO scheduling.

However, we also expect that lower-priority jobs may experience longer waiting times because higher-priority jobs will be executed first.

Our goal is not to prove that priority scheduling is always better. Instead, we want to measure how changing the scheduling policy affects different groups of jobs.

The scheduler will control the order in which batch jobs are launched. It will not modify the Linux kernel's CPU scheduling algorithm.

Algorithm Design and Complexity

The FIFO scheduler will use a standard queue.

For a FIFO queue, inserting a job and removing the next job both take O(1) time.

The priority-based scheduler will use a binary heap.

For a binary heap containing n jobs:

- Inserting a new job takes O(log n) time.
- Removing the highest-priority job takes O(log n) time.
- Checking the highest-priority job takes O(1) time.

The binary heap allows the scheduler to maintain priority ordering without sorting the entire job list after every insertion.

Although FIFO has lower queue operation costs, it cannot select jobs based on their priority levels.

The main difference between these two approaches is the job selection algorithm, rather than the execution speed of individual programs.

4. Research Question

Our primary research question is:

Does priority-based batch scheduling reduce the average waiting time of high-priority jobs compared with FIFO scheduling in a UNIX shell?

We will also investigate the following questions:

1. How does priority-based scheduling affect the waiting time of low-priority jobs?

2. How do the two scheduling policies compare in average turnaround time?

3. How does the number of submitted jobs affect scheduling behavior?

We will evaluate the following metrics:

- Average waiting time
- Average turnaround time
- Waiting time by priority level
- Job completion order
- Scheduling fairness

Waiting time:

Waiting Time = Start Time - Submission Time

Turnaround time:

Turnaround Time = Completion Time - Submission Time

For our experiments, all jobs will be submitted before batch execution begins. We will use the batch start time as the common reference point for calculating waiting time.

The same workloads will be used for both scheduling policies to ensure a fair comparison.

5. Proposed Scope

We plan to complete the following five technical tasks:

1. Implement a basic UNIX shell using fork(), execvp(), and waitpid() to support command execution and process management.

2. Add support for input/output redirection, pipelines, and background process execution.

3. Implement a FIFO batch scheduling system that stores jobs in submission order and executes them one at a time.

4. Implement a priority-based batch scheduler using a binary heap, with FIFO ordering for jobs that have the same priority.

5. Develop a benchmarking framework to measure waiting time, turnaround time, job completion order, and scheduling fairness under different workloads.

These tasks will allow us to build a working shell while keeping the research extension focused on one specific scheduling problem.

6. Experimental Design

Experimental Environment

All experiments will be conducted on the same Ubuntu virtual machine.

Both scheduling policies will execute the same sets of batch jobs under similar system conditions.

We will execute one batch job at a time so that our results focus on job selection order rather than competition between concurrently running jobs.

All jobs will be submitted before batch execution begins.

Workload Configuration

We plan to evaluate three workload sizes:

Workload	Number of jobs	Purpose
Small	10	Verify basic scheduling behavior
Medium	50	Compare waiting times across priority levels
Large	100	Observe scheduling behavior with more jobs

Each workload will contain a mixture of short, medium, and long-running jobs.

We will assign jobs to three priority levels:

- High priority
- Medium priority
- Low priority

The workloads will include jobs with different execution times and priorities so that we can observe how scheduling order affects waiting time.

For each workload, we will keep the job commands, priority assignments, and submission order identical when comparing FIFO and priority-based scheduling.

Evaluation Methodology

For each experiment, we will record:

- Job ID
- Job priority
- Submission order
- Job start time
- Job completion time
- Job execution order

We will use a common batch start time as the reference point for waiting-time calculations. Since all jobs are submitted before execution begins, this allows us to compare how long each job waits after the batch becomes ready.

We will calculate the average waiting time and average turnaround time for each scheduling policy.

We will also calculate waiting times separately for high-, medium-, and low-priority jobs.

Each workload will be tested under both scheduling policies using repeated trials.

We will compare the results using tables and graphs.

Scheduling Fairness

Priority-based scheduling may improve waiting times for high-priority jobs while increasing delays for low-priority jobs.

We will evaluate scheduling fairness by comparing average waiting times across different priority levels.

We will also examine whether low-priority jobs experience significantly longer delays under priority-based scheduling than under FIFO scheduling.

Because all jobs are submitted before execution begins, every job will eventually be executed. Therefore, our experiments will focus on waiting-time differences rather than indefinite starvation.

Limitations and Expected Trade-offs

Our scheduler will operate in user space and control the order in which batch jobs are launched.

It will not modify the Linux kernel scheduler or control CPU time allocation directly.

The batch scheduler will be non-preemptive. Once a job starts running, it will finish before the next job is selected.

We will not investigate dynamically arriving jobs during batch execution in the initial implementation.

We also do not expect priority-based scheduling to improve every performance metric.

Because both policies execute the same jobs sequentially, changing the scheduling order does not reduce the total amount of work performed.

Priority scheduling may reduce the waiting time of high-priority jobs, but this improvement may come at the cost of longer waiting times for lower-priority jobs.

Our experiments will help us understand these trade-offs and determine whether priority-based scheduling provides measurable benefits for high-priority batch jobs.